Understanding Delta Kernel

A Simple Guide to the Core Library Behind Delta Lake.
Structure, Purpose, Philosophy, and Usage.

---

1. What is Delta Kernel?

Delta Kernel is a Rust library that contains the entire Delta Lake protocol in one place. Think of it as the brain of Delta Lake. It knows how to read the transaction log, how to understand checkpoints, how to skip files that are not needed, and how to build a snapshot of a table at any point in time. But it does NOT do any I/O (reading or writing files) by itself. Instead, it asks another component, called the Engine, to do the actual file work for it. This is the most important idea in Delta Kernel.

Before Delta Kernel existed, every engine (Spark, Flink, DuckDB, ClickHouse, and others) had to write its own code to understand the Delta Lake protocol. This meant a lot of duplicated work, and each engine had a slightly different (and sometimes buggy) implementation. Delta Kernel solves this problem by giving everyone one single, correct, and well-tested library for the protocol. Now, when a new feature is added to the Delta protocol, every engine gets it just by updating their Delta Kernel dependency.

1.1 Why Do We Need It?

Imagine you have 10 different programs that all need to read Delta Lake tables. Without Delta Kernel, each program must understand how to: parse JSON commit files, read checkpoints, handle protocol versions, apply schema evolution, manage deletion vectors, and perform data skipping. That is a huge amount of complex code to write and maintain in 10 different places. With Delta Kernel, all of this logic lives in one library. Each program only needs to provide a simple Engine implementation that says how to read files and evaluate expressions. The Kernel handles everything else.

2. The Philosophy Behind Delta Kernel

The Delta Kernel was created with a clear set of design principles. These principles guide every decision in the library and are the reason why it works so well with so many different engines.

1. Protocol is separate from Engine: The Kernel knows the Delta Lake protocol. The Engine knows how to read and write files. These two things should never mix. The Kernel tells the Engine what to do, and the Engine does it. This separation is the core philosophy.

2. No async in the core: The Kernel itself is completely synchronous (no async/await). It never forces you to use tokio or any other async runtime. Only the DefaultEngine uses async internally, and that is an optional feature that you can choose to use or not.

3. No Arrow dependency by default: The Kernel does not assume you use Apache Arrow for your data format. You can use any columnar format you want. Arrow support comes as an optional feature flag.

4. Clear I/O boundaries: Every method in the Kernel that needs to interact with the outside world clearly shows it. Methods that need I/O use names like fetch, read, or list. If a method does not have these names, you can be sure it does not touch the filesystem.

5. Builder-style APIs: The Kernel uses the builder pattern for complex objects like Snapshot and Scan. This makes the code easy to read and configure step by step.

6. Zero-copy data access: The Kernel uses a visitor pattern (GetData and RowVisitor) to access data. It never copies your data into its own structures. It visits your data where it lives.

7. Engine-agnostic by design: The Kernel does not care if your engine uses Arrow, Parquet, a custom format, or something else. As long as you implement the Engine trait, the Kernel works with you.

8. Minimal dependencies: The core Kernel has very few dependencies. Heavy libraries like Arrow, Parquet, and object_store are only pulled in when you enable specific feature flags.

3. How the Architecture Works

The Delta Kernel has a layered architecture. Each layer has a clear job, and layers only talk to the layers directly above or below them. This makes the system easy to understand and easy to change. Let us look at each layer from top to bottom.

3.1 The Layer Diagram

At the top is your Compute Engine (Spark, DuckDB, Polars, or your own program). Below that is your Connector code, which calls the Kernel. The Kernel is the middle layer that handles all Delta protocol logic. Below the Kernel is the Engine trait, which is the boundary between protocol logic and I/O. At the bottom is the DefaultEngine (or your custom engine) and the actual storage (S3, local disk, Azure, GCS).

Layer	What It Does	Who Provides It

Compute Engine	Runs queries and processes data	You (or Spark, DuckDB, etc.)
Connector	Connects engine to Delta Kernel	You write this
Delta Kernel	Protocol logic, snapshots, scans	delta-kernel-rs crate
Engine trait	I/O and expression abstraction	Defined by Kernel, implemented by you
DefaultEngine	Arrow + object_store + tokio	Optional, comes with Kernel
Storage	Actual files on disk or cloud	S3, Azure, GCS, Local FS

3.2 The Engine Trait - The Most Important Boundary

The Engine trait is the most important concept in Delta Kernel. It is the boundary between what the Kernel knows (the Delta protocol) and what the outside world knows (how to read files, parse JSON, evaluate expressions). The Engine trait has four main parts:

Trait Part	Job	What It Does
EvaluationHandler	Evaluate expressions	Check if a value matches a filter, compute new values
StorageHandler	File system operations	List files, read files, write files, copy files
JsonHandler	JSON parsing	Read and parse JSON commit files from the transaction log
ParquetHandler	Parquet operations	Read and write Parquet checkpoint and data files

When the Kernel needs to do something like read a commit file, it calls StorageHandler to get the file bytes, then calls JsonHandler to parse those bytes into structured data. The Kernel never reads or writes anything directly. It always goes through the Engine. This is what makes the Kernel so flexible and easy to integrate with any system.

4. How Data Flows Through the System

Let us walk through what happens when you want to read data from a Delta Lake table. This will help you understand how all the pieces fit together.

Step 1: Build a Snapshot

First, the Kernel builds a Snapshot. A Snapshot is a complete view of the table at a specific version. To build it, the Kernel asks the StorageHandler to list files in the _delta_log/ directory. It looks for the _last_checkpoint hint file to know where to start. Then it reads the checkpoint (using ParquetHandler) and any commit files that come after the checkpoint (using JsonHandler). From these files, it learns the table schema, the protocol version, and the list of active data files (by combining Add and Remove actions).

Step 2: Build a Scan

Next, the Kernel builds a Scan from the Snapshot. A Scan represents a read operation with specific columns and an optional filter (predicate). When you call scan.scan_metadata(), the Kernel does several things: it replays the log to find all active files, it applies data skipping (using file statistics to skip files that cannot match your filter), and it prepares the transform expressions needed to read the correct columns.

Step 3: Execute the Scan

Finally, scan.execute() reads the actual data files. The Kernel tells the ParquetHandler which files to read and which columns to project. Each file produces one batch of data. You get these batches one by one and can process them however you want. If the table uses deletion vectors, the Kernel also handles those - it reads the deletion vector files and removes the deleted rows from the results.

4.1 The Complete Flow (in Code)

```
use delta_kernel::engine::default::DefaultEngine;
use delta_kernel::engine::default::storage::store_from_url;
use delta_kernel::{Snapshot};

// Step 1: Parse the table location
let url = delta_kernel::try_parse_uri("./my_table")?;

// Step 2: Create the engine
let store = store_from_url(&url)?;
let engine = DefaultEngine::builder(store).build();

// Step 3: Build a snapshot (point-in-time view)
let snapshot = Snapshot::builder_for(url)
  .build(&engine)?;

// Step 4: Build and execute a scan
let scan = snapshot.scan_builder().build()?;
```

```
for result in scan.execute(engine)? {
  let batch = result?;
  // Process each batch of data
}
```

5. Key Concepts Explained Simply

5.1 Snapshot

A Snapshot is a frozen picture of a table at a specific version. It contains everything you need to know about the table at that point: the schema (what columns exist and their types), the protocol version (what features are enabled), and the list of active data files. Once a Snapshot is built, it never changes. If the table gets new commits, you need to build a new Snapshot. You can also build a Snapshot at a specific old version (time travel). Snapshots can be built incrementally - if you already have one, you can update it to the newest version without starting from scratch.

5.2 Scan

A Scan is a plan for reading data from a table. You build it from a Snapshot by specifying which columns you want and optionally a filter predicate. The Scan uses data skipping to avoid reading files that cannot contain matching data. This is done by checking the min/max statistics stored in each file's metadata. If your filter says value > 100 and a file's max value is 50, that file is skipped entirely. This can make queries much faster.

5.3 Log Segment

A Log Segment is a contiguous section of the transaction log. It guarantees that there are no gaps in the commit versions and that checkpoints are complete. The Kernel uses Log Segments to efficiently determine which files it needs to read. If a checkpoint exists, the Log Segment only includes commits after the checkpoint, which avoids reading many small JSON files.

5.4 EngineData and the Visitor Pattern

EngineData is an opaque (hidden) data buffer. The Kernel does not know or care what is inside EngineData. It could be Arrow RecordBatches, or it could be some completely custom data format. When the Kernel needs to read a value from EngineData, it uses a visitor pattern. It calls GetData.get() with a path like ["add", "path"] to get a specific value. The Engine provides the actual implementation. This is how the Kernel stays engine-agnostic - it never touches your data directly.

5.5 Data Skipping

Data Skipping is a powerful optimization. Every data file in a Delta table has statistics (min value, max value, and null count for each column). When you run a query with a filter, the Kernel rewrites the filter into a form that can be evaluated against these statistics. For example, if your query filters x > 100, the Kernel creates a skipping predicate: max_x > 100. If a file's max_x is 50, the file is skipped. This happens before any actual data is read, so it saves a lot of time on large tables.

6. Inside the Crate Structure

The Delta Kernel project has several crates, each with a clear purpose. The main crate is delta_kernel, which contains all the protocol logic. Let us look at the important modules inside it.

Module	What It Contains	Why It Matters
snapshot/	Snapshot builder and management	Point-in-time table views
scan/	Scan builder, data skipping, log replay	Reading data efficiently
engine/	Engine trait + DefaultEngine	I/O abstraction boundary
actions/	Add, Remove, Metadata, Protocol, etc.	Delta protocol action types
schema/	Kernel's own type system	Column types and schema handling
expressions/	Predicates, column refs, scalars	Filter and expression logic
log_segment/	LogSegment and checkpoint handling	Efficient log file management
log_replay	Action replay from commits	Building state from JSON files
partition/	Partition handling	Partition column management
transaction/	Transaction for the write path	Writing data to tables
history_manager/	Table version history	Listing commits and versions
checkpoint/	Checkpoint writer (V1 and V2)	Creating checkpoint files

7. DefaultEngine vs Custom Engine

You have two choices when working with Delta Kernel. You can use the built-in DefaultEngine, or you can write your own Engine implementation. Let us compare them to help you decide.

Aspect	DefaultEngine	Custom Engine
Setup effort	Easy - just create and use	Hard - implement all 4 traits
Data format	Apache Arrow	Any format you want
I/O backend	object_store (S3, Azure, GCS, local)	Your own storage system
Async runtime	Uses tokio internally	No runtime required in Kernel
JSON parsing	Arrow JSON reader	Your own JSON parser
Parquet	Arrow Parquet reader/writer	Your own Parquet reader
Best for	Quick start, Rust projects	Non-Arrow engines (C++, Java, etc.)

The DefaultEngine is perfect for most Rust projects. It uses Apache Arrow for data, the object_store crate for I/O, and tokio for async operations. You get all of this by enabling just one feature flag: default-engine-rustls. A custom Engine is needed when you are building a connector for a non-Rust engine (like ClickHouse in C++, or DuckDB in C++), or when you have special storage needs that object_store cannot handle.

8. How Delta Kernel and delta-rs Are Different

It is easy to confuse Delta Kernel and delta-rs because they both work with Delta Lake in Rust. But they serve very different purposes. Delta Kernel is the low-level protocol library. It only knows the Delta protocol - how to read logs, build snapshots, and plan scans. delta-rs is a higher-level library that uses Delta Kernel internally and adds many more features on top.

Feature	Delta Kernel	delta-rs
Purpose	Protocol logic only	Full Delta Lake operations
Python bindings	No	Yes (via PyO3/maturin)
SQL queries	No	Yes (via DataFusion)
MERGE/UPDATE/DELETE	Experimental	Full support
OPTIMIZE/VACUUM	No	Yes
Arrow dependency	Optional	Required
Async runtime	Optional	Required (tokio)
I/O operations	Delegated to Engine	Built-in
Target users	Engine builders	Application developers

In simple terms: if you want to build a Delta Lake connector for a new engine, use Delta Kernel. If you want to build an application that reads and writes Delta tables in Rust, use delta-rs. The delta-rs library uses Delta Kernel internally for all protocol-level operations, so you get the best of both worlds.

9. Setting Up Delta Kernel in Your Project

Getting started with Delta Kernel is very simple. Add it to your Cargo.toml with the features you need. The core library has no default features, so you only pay for what you use.

```
# Cargo.toml - Minimal (no I/O, no Arrow)
[dependencies]
delta_kernel = "0.23"

# With DefaultEngine (Arrow + object_store + rustls)
[dependencies]
delta_kernel = { version = "0.23", features = [
  "default-engine-rustls",
  "arrow",
] }

# Or use native-tls instead of rustls
delta_kernel = { version = "0.23", features = [
  "default-engine-native-tls",
  "arrow",
] }
```

The main feature flags are: default-engine-rustls (DefaultEngine with Rust TLS), default-engine-native-tls (DefaultEngine with system TLS), arrow (Arrow data format support), arrow-conversion (convert between Arrow and Kernel types), and arrow-expression (Arrow-based expression evaluation). The core Kernel works without any of these - you only need them for

specific use cases.

10. Key APIs You Will Use

These are the main types and methods you will work with when using Delta Kernel directly. Each one has a clear purpose and follows the builder pattern.

Type / Method	What It Does	When to Use
Snapshot::builder_for(url)	Build snapshot from table path	Starting any table operation
.at_version(n)	Time travel to version n	Reading old data
.build(&engine;)	Execute the snapshot build	After configuring the builder
snapshot.scan_builder()	Create a scan from snapshot	Preparing to read data
.with_predicate(pred)	Set filter for the scan	Querying subset of data
.with_schema(cols)	Select specific columns	Column projection
scan.execute(engine)	Read data as batches	Getting actual data
scan.scan_metadata(engine)	Get scan metadata first	When you need file info
try_parse_uri(path)	Parse a path string to URL	Before building a snapshot

11. Summary: The Big Picture

Delta Kernel is the foundation of the Delta Lake ecosystem. It gives every engine a single, correct, and well-tested implementation of the Delta Lake protocol. By keeping the protocol logic separate from I/O and data format concerns, it works with any engine, any data format, and any programming language (through the C FFI). The philosophy is simple: the Kernel knows the protocol, and you provide the Engine that knows your system. Together, they give you full access to Delta Lake tables with minimal code.

1. Delta Kernel = protocol logic (read logs, build snapshots, plan scans).
2. Engine trait = I/O and expression evaluation (your job to implement).
3. DefaultEngine = ready-to-use Engine with Arrow + object_store + tokio.
4. Snapshot = frozen view of the table at a specific version.
5. Scan = plan for reading data with columns and filters.
6. Data Skipping = skip files using min/max statistics.
7. EngineData = opaque data buffer accessed via visitor pattern.
8. delta-rs = higher-level library that uses Delta Kernel internally.
